

DSST389: Semester I Function Reference (Ch. 1–9)

The Polars and plotnine functions from first semester, grouped by the task you are trying to do. The red lines are the ideas worth memorizing; everything else is lookup.

Polars

The One Big Idea

► *An expression like `c.hdi > 0.9` is a **description**, not data. It produces values only inside a **context** that runs it on a real `DataFrame`.*

- contexts: `.select .filter .with_columns .sort .group_by().agg .over()`
- the same expression works unchanged in any of them
- `c.col` — refer to a column by name
- `c("odd name")` — names that aren't valid identifiers

Read & Inspect

- `pl.read_csv(path) / pl.read_excel(path)`
- `df` or `df.print(n)` — show `n` rows (course helper; default 8)
- `df.schema` — each column and its type
- `len(df)` — number of rows
- `df.describe()` — per-column summary stats at a glance
- `df.pipe(f, ...)` — run any function as a chain step; `df.pipe(f) = f(df)`

Choosing Rows

- `.sort(cols, descending=False)`
- `.filter(cond)` — keep `True` rows; `null` drops like `False`
- comparisons: `== != < <= > >=`
- `(a) & (b)` (and) `(a) | (b)` (or) — parenthesize each side
- `.is_in([...])`
- `.head(n=)` `.tail(n=)`
- `.sample(n=)` or `(fraction=)`
- `.unique(subset=[...], keep="first"/"last")` — one row per group; `.sort` first to control which

Choosing & Making Columns

- `.select(...)` — keep only these columns
- `.drop(...)` — remove these columns
- `.rename({"old": "new"})`
- `.with_columns(name = expr, ...)` — keep all columns, add/overwrite
- `pl.when(c).then(a).when(c2).then(b).otherwise(d)` — first match wins
- `pl.lit("text")` — a literal value, not a column name
- `.alias("name")` — older way to name a column inside `with_columns/agg`; still common in docs

Types & Missing Data

- core types: `str`, `i64`, `f64`, `bool`
- `.cast(pl.Int64/Float64/ String/Boolean, strict=False)` — unconvertible → `null`
- `null` = missing; `NaN` = broken math (`f64` only)
- `.is_null()` / `.is_nan()`
- `.fill_null(v)` / `.fill_nan(v)`
- `.drop_nulls()` / `.drop_nans()`

- `nulls` propagate: arithmetic *and* comparison vs `null` → `null` (aggregations skip them instead)
- `.str.to_uppercase()` `.str.to_date()` `.str.split("sep")`

Summaries: `group_by` + `agg`

► *Aggregation **reduces** each group to one row.*

- `.group_by(cols, maintain_order=False)` — then chain `.agg`, `.head`, `.tail`, or a shortcut like `.mean()`
- `.agg(name = expr, ...)` — one row per group
- whole frame (no groups): `.select(name = expr.mean())` — `agg` needs a `group_by`; `select` does not
- `.mean()` `.median()` `.min()` `.max()` `.sum()` `.quantile(q)` `.first()` `.last()` `.any()` `.all()`
- `pl.corr(a, b)`
- counting: `pl.len()` rows/group · `col.count()` non-null · `col.n_unique()` distinct (`null` counts as a value)

Windows: `.over()`

► *A window computes the same summaries but **keeps** every row, writing the group's answer onto each one.*

- `expr.over(cols)` — goes inside `with_columns` or `filter`
- ordered (`.sort` first): `.shift(k)` (neg = forward) · `.diff()` · `.cum_sum()` · `.rank(descending=)` · `.rolling_mean(window_size=k)`
- `.forward_fill()` / `.backward_fill()`
- scope ordered ops with `.over(group)` so they never cross a group boundary

Joins

► *Put the foreign key on the left, the primary key on the right.*

- `left.join(right, on=col, how="inner"/"left"/"right"/"full")`
- `on=[c.a, c.b]` `left_on=` / `right_on=suffix="_right"`
- `how="semi"` keep matched (no new cols) `how="anti"` keep unmatched
- safety check before joining: `right.select(c.key).n_unique() == len(right)`
- `.join_asof(right, on=col, by=, strategy="backward"/"forward"/"nearest")` — both sorted first
- `.join_where(right, expr)` — pairs by condition; shared names get `_right`

Reshape & Stack

- `.pivot(index=, on=, values=)` — long → wide
- `.unpivot(index=, variable_name=, value_name=)` — wide → long
- `pivot/unpivot` want column *names as strings*, not `c.`
- `.str.split("sep")` then `.explode(col)` — one list cell → one row each
- `pl.concat([df1, df2])` — stack rows of two frames

The One Big Idea

► *A plot = data + aes() mappings + geoms, assembled by piping.*

- `df.pipe(ggplot, aes(x, y)) .geom_*`
- inside `aes()`: mapped to a column
- outside `aes()` but still in the geom: a fixed value
- put fixed arguments *after* the `aes()` call
- shared aesthetics: `color`, `fill`, `alpha`, `size`, `shape`, `group`

Geoms: One Shape Per Row

- `.geom_point()` `.geom_line()`
- `.geom_text(aes(label=c.col), size=, nudge_y=)`
- `.geom_col()` — you supply the bar height (needs `x`, `y`)
- `.geom_*(data=other_df)` — different data for one layer
- `.coord_flip()` `.coord_fixed()`

Geoms: Computed From Data

► *Each of these runs a `group_by+agg` for you before drawing.*

- `.geom_bar()` — counts rows per category itself (vs `geom_col`, where you give the height)
- `.geom_histogram(binwidth=, boundary=)` or `(bins=)`
- `.geom_density(adjust=1)`
- `.geom_boxplot()` — 25/50/75 box, whiskers, outliers
- `.geom_smooth(method="lm", se=False)` — line of best fit
- split by group: add a `color=` or `group=` aesthetic (a few groups only)

Ordering Categories

► *A categorical axis follows the row order in the data — so `.sort()` before plotting.*

- value already a column: `.sort(c.n)` then plot
- order by a group summary: `.sort(pl.len().over(c.cat))` or `.sort(c.num.median().over(c.cat))`
- treat a numeric axis as categories (e.g. years): `.cast(pl.String)`

Scales

- `scale_color_cmap_d(cmap_name="plasma")` — viridis family, discrete (default)
- `scale_color_cmap()` — viridis, continuous
- `scale_color_brewer(type="qual", palette=)`
- `scale_color_manual(values={...})`
- `scale_color_identity()` — use the column's own color values
- `scale_size_area()`
- `scale_x_log10()` / `scale_y_log10()`
- `scale_x_continuous(breaks=breaks_width(w), limits=(lo, hi))` — `None` leaves one end automatic

Facets, Labels, Themes

- `.facet_wrap("col") .facet_grid("row", "col")`
- `scales="free_x"/"free_y"/"free"` — independent panel ranges (rare; breaks comparison)
- `show_legend=False` — inside a geom, hide its legend
- `.labs(x=, y=, color=, title=, subtitle=, caption=)`
- `theme_minimal()` (default here) `theme_set(theme_minimal())`